

P Loss Curves for Complementary Strategies

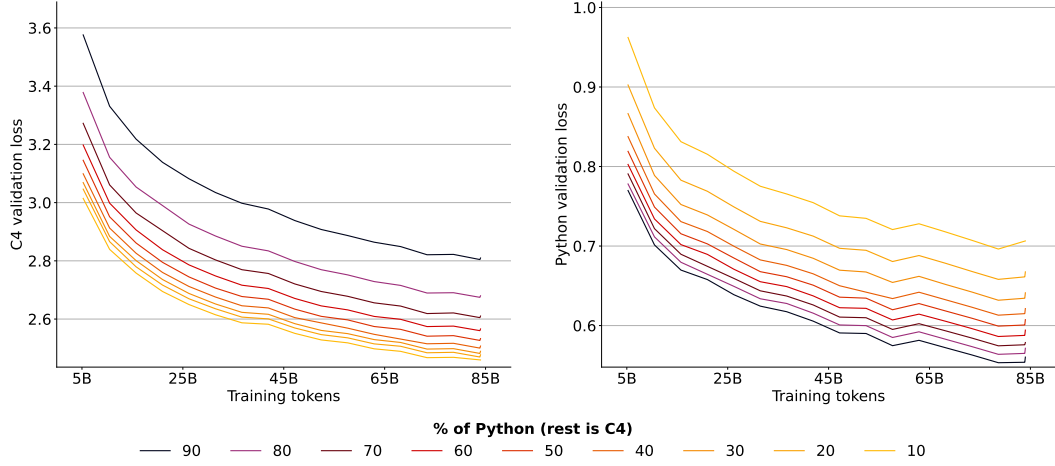


Figure 19: Validation loss of models trained on a mix of natural language (C4) and Python data.

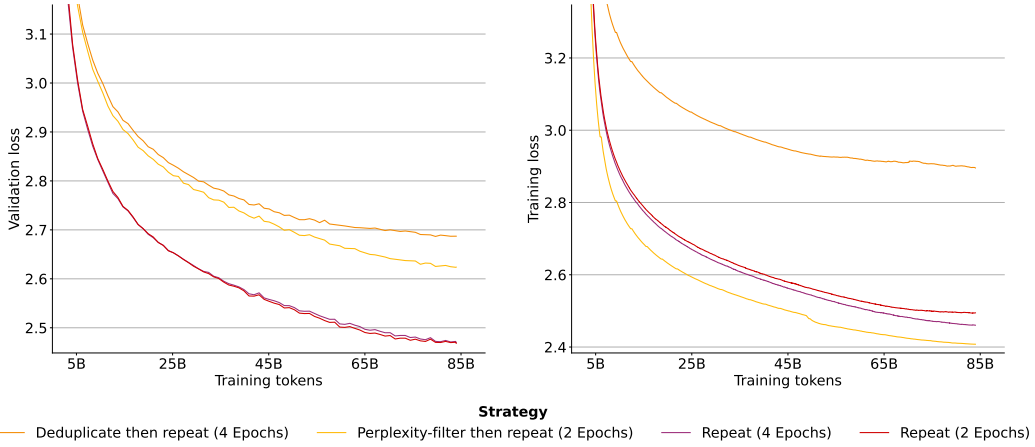


Figure 20: Validation and training loss of models trained with different data strategies. Training loss is smoothed with exponential moving average smoothing and a weight of 0.999. Downstream performance of the models is in Figure 6.

To compare complementary data strategies in §7, we have used downstream performance on natural language tasks detailed in Appendix K instead of loss. This is because validation loss gives an unfair advantage to models trained on a larger fraction of data from the same distribution. For example, when making up for missing natural language data with code, models that are trained on more code will have better validation loss on code data while having worse loss on the natural language data as seen in Figure 19: The model pre-trained on 90% of Python code data and 10% of C4 has the highest C4 validation loss, but the lowest Python validation loss.

Models trained on deduplicated or perplexity-filtered data have higher validation loss as the held-out validation data has not gone through the same filtering steps. Thus, its distribution more closely resembles the training data of models trained on the unfiltered data resulting in worse validation loss for the two filtering strategies in Figure 20 (left). Meanwhile, for training loss in Figure 20 (right) the model trained on perplexity-filtered data has the lowest loss. Its training data has been filtered to the top 25% of examples with the lowest perplexity (Appendix N) thus high loss examples have been explicitly filtered out from the training data resulting in low training loss. The model trained on deduplicated data has the highest validation and training loss. This is because commonly

repeated sequences have been filtered out from its training data. Thus, when encountering these common sequences in the unfiltered validation set, its loss is comparatively high as other models have likely simply memorized them. Similarly, fewer repeated sequences during training results in higher training loss as unseen sequences are harder to predict.

Q Limitations and Future Work

Repeating fractions of the data In this work we focus on repeating the entire unique dataset for several epochs. Alternatively, one can repeat only a fraction of the dataset. For example, repeating 10% of the dataset for 10 epochs while repeating the rest only for a single epoch as done by Hernandez et al. [40]. To predict loss in that scenario, one may need to adapt our scaling laws with an additional parameter to account for the fraction that is repeated and possibly a parameter that captures at what point in training the data is repeated. Repeating earlier in training when most model weights are still randomly initialized is likely to cause less damage than later in training. Adapting our parametric fit to make concrete scaling predictions for such scenarios is an exciting future research direction.

Sensitivity to hyperparameters The returns from additional epochs may heavily depend on hyperparameters such as learning rate, dropout, or the optimizer choice. It is likely that increasing the learning rate, for example, would lead to diminishing returns from additional epochs kicking in earlier. In this work, we have fixed most hyperparameters to commonly used values for the training of LLMs and leave such explorations to future work.

Other datasets The optimal data strategy is dependent on the dataset at hand and we cannot give universally applicable filtering recommendations. By looking into C4 and OSCAR, we have covered two of the most commonly used English text datasets. Our findings on both datasets were overall in agreement with each other. We have highlighted some of the differences, such as deduplication being more effective on OSCAR due to it being more noisy than C4. Further, we have focused on large-scale pre-training datasets. There is a lot of research on the optimal fine-tuning dataset and methodology for LLMs [94, 62, 128, 85, 117, 68, 116, 134, 115, 36, 125, 72, 63]. More investigations of resolving data-constraints when fine-tuning LLMs may be of interest for future work.

Other modalities or architectures Our work focuses on text datasets and uses the GPT transformer architecture [88]. Prior work has experimented with many variations to the GPT or transformer architecture [27, 104, 96], as well as scaling laws for non-text datasets [1]. Overall, variations of the GPT or transformer architecture have proven very robust and generalizable to other domains [43, 18, 70, 66, 71, 104, 26]. Nonetheless, it may be of interest for future work to test the applicability of our findings in this work to different data modalities or model architectures.

Other strategies There are numerous strategies to solve data constraints not covered in this work that are worth exploring. Like we have shown for Python, future research may consider to what extent augmenting with a natural language (e.g. Chinese) improves performance in another language (e.g. English) and what is the best language to choose [61, 124]. Similarly, while we have looked at deduplication and perplexity filtering, other filtering strategies, such as popularity-based filters [3, 133] and toxicity filters [33, 38, 64, 87, 86] are worth exploring.

R Contributions

Niklas Muennighoff led experiments, analysis, writing, and the overall project. He implemented, trained and evaluated all models.

Alexander M. Rush contributed to framing, results analysis, and paper writing.

Boaz Barak contributed to formal and experimental analysis as well as paper writing.

Teven Le Scao provided guidance, led data choices and preprocessing, and contributed to framing and writing.

Aleksandra Piktus created perplexity and deduplication datasets and contributed to writing.

Nouamane Tazi contributed to enabling high-performance training on AMD hardware.

Sampo Pyysalo contributed to enabling high-performance training and early repetition experiments.

Thomas Wolf provided guidance on experimental design and contributed to paper writing.

Colin Raffel provided guidance on experimental design and contributed to paper writing.

S Hyperparameters and Setup

For all training runs we use 1% of tokens for linear warm-up of the learning rate to a maximum learning rate of $2e-4$ that is decayed to $2e-5$ following a cosine schedule. We use a batch size of 256 for models with fewer than 2 billion parameters, 512 for models with 2 - 5 billion parameters and 1024 for models with more than 5 billion parameters. All models are trained in bfloat16 precision using the Adam optimizer [48] with $\epsilon = 1e-8$, $\beta_1 = 0.9$. For β_2 , we found a value of 0.95 to result in slightly lower final loss and fewer loss spikes than the default value of 0.999 in implementations such as PyTorch. However, except for models with FLOP budgets of $C = 9.3 \times 10^{20}$ and 2.1×10^{21} , we always use $\beta_2 = 0.999$. We use a dropout rate of 0.1, a weight decay rate of 0.1 and clip gradients at 1.0. These hyperparameter choices are largely based on prior work [42, 110] and performance on test runs. As none of our hyperparameter choices is particularly exotic, we expect our setup to generalize to many other setups. In Table 15 we list the model architectures we use. They are an extended version of the architectures from [42]. We calculate model parameters following [78], which includes embedding parameters:

$$P = 12lh^2 \left(1 + \frac{13}{12h} + \frac{V + s}{12lh} \right) \quad (23)$$

where P is the final parameter count, l are layers, h is the hidden dimension, $V = 50257$ the vocabulary size and $s = 2048$ the sequence length. We find the parameter counts reported in Chinchilla [42] to be significantly different than our calculations, especially at larger scales. Part of this difference comes from their smaller vocabulary size of $V = 32000$. We report both our and the parameter counts of Chinchilla in Table 15, but we use our parameter estimates everywhere in this work. Further, we have corrected the number of heads of the 3,530 and 4,084 million parameter models from [42] to obey the relationship $d_{\text{model}} = kv_size \cdot n_heads$.

To train our models, we have forked the Megatron-DeepSpeed [91, 99] framework and adapted it for ROCm to enable training on AMD GPUs. We have made our training code publicly available at <https://github.com/TurkuNLP/Megatron-DeepSpeed>. Models are trained using data, tensor and pipeline parallelism on up to 256 AMD Instinct MI250X GPUs distributed across up to 64 nodes on the LUMI supercomputer located in Finland. As of June 2023, LUMI is the largest supercomputer in Europe and ranks third worldwide with a performance of around 310 PFLOPs.⁵ We trained models in parallel using up to 2,200 nodes at a single point in time (equivalent to around 8,800 GPUs or 17,600 GCDs or 86% of all GPUs on LUMI). We have used a total of around 3 million GPU hours. The cluster is powered 100% by renewable energy (hydroelectricity) and its waste heat is used for heating the nearby city reducing the city’s carbon emissions by up to 20%. Thanks to the low temperatures in Finland, relatively little cooling for the cluster is required further reducing its impact on the environment. As of June 2023, it ranks as the seventh greenest supercomputer.⁶

⁵<https://www.top500.org/lists/top500/2023/06/>

⁶<https://www.top500.org/lists/green500/2023/06/>